

RabbitBAM: Accelerating BAM File Manipulation on Multi-Core Platforms

Lifeng Yan , Zhan Zhao, Zekun Yin , Tong Zhang , Yang Yang , Fangjin Zhu, Xiaohui Duan, Bertil Schmidt , *Senior Member, IEEE*, and Weiguo Liu 

Abstract—With the continuous advancement of sequencing technology, the scale of biological data has rapidly increased. BAM format, widely used for storing aligned sequence data, is very popular due to its ease of use and good compression ratio. However, existing BAM-format file I/O libraries often fail to fully leverage the computational power of modern multi-core platforms, resulting in low CPU utilization. To address this, we introduce RabbitBAM, a fast BAM-format file I/O library. RabbitBAM employs pre-parsing and parallel parsing techniques to eliminate parsing bottlenecks and improve parallel efficiency. Additionally, we optimize multi-threaded data handling through the use of dedicated lock-free queues and memory pools. RabbitBAM achieves 2.1-3.3x speedups on next-generation sequencing data and 1-2.2x speedups on third-generation sequencing data compared to state-of-the-art SAMtools (HTSlib). We also present two case studies (BAM file quality control and sorting) using RabbitBAM, demonstrating 1.4-2.4x speedups compared to other implementations.

Index Terms—BAM format, high-performance bio-computing, I/O library, pre-parsing, sequencing data.

I. INTRODUCTION

With the rapid advancement of sequencing technologies, the volume of data acquired by researchers has grown at an unprecedented rate [1], [2]. For instance, an Illumina NovaSeq 6000 sequencer can generate 3 TB of data per day, and in the field of metagenomics, sequencing just one gram of soil sample can yield 50 TB of raw data [3]. This enormous growth in data volume poses significant challenges to the efficiency of corresponding sequencing data processing software [4], [5].

The majority of sequencing data analysis workflows begin with preprocessing the raw reads using quality control tools such as fastp [6]. Alignment tools like minimap2 [7] are then employed to map the sequencing fragments to a reference genome, producing output files in BAM (Binary Alignment/Map) or SAM (Sequence Alignment/Map) format. These BAM files subsequently undergo metadata modification, sorting, and duplicate marking. The resulting data is then used for downstream analyses such as germline and somatic variant calling [8].

Received 5 November 2024; revised 1 July 2025; accepted 15 July 2025. Date of publication 18 July 2025; date of current version 9 October 2025. This work was supported in part by NSFC under Grant 62102231, in part by Shandong Provincial Natural Science Foundation under Grant ZR2021QF089, in part by Engineering Research Center of Digital Media Technology, Ministry of Education, China, and in part by DFG RMaP project C01. (*Lifeng Yan and Zhan Zhao are Co-first authors.*) (*Corresponding authors: Zekun Yin; Weiguo Liu.*)

Lifeng Yan, Zhan Zhao, Zekun Yin, Tong Zhang, Yang Yang, Fangjin Zhu, Xiaohui Duan, and Weiguo Liu are with the School of Software, Shandong University, Jinan 250100, China (e-mail: lifeng.yan@mail.sdu.edu.cn; zhaozhan@mail.sdu.edu.cn; zekun.yin@sdu.edu.cn; tong.z@mail.sdu.edu.cn; yang.yang@mail.sdu.edu.cn; zhufj@sdu.edu.cn; sunrise.duan@sdu.edu.cn; weiguo.liu@sdu.edu.cn).

Bertil Schmidt is with the Institute for Computer Science, Johannes Gutenberg University, 55122 Mainz, Germany (e-mail: bertil.schmidt@uni-mainz.de).

RabbitBAM and case studies are freely available at <https://github.com/RabbitBio>, with usage documentation at <https://rabbitbam.readthedocs.io/en/latest>.

Digital Object Identifier 10.1109/TCBBIO.2025.3590412

In addition, other applications — such as large-scale RNA-seq analysis for expression quantification and fusion gene detection — also often involve extensive BAM file operations [9], [10], often requiring repeated traversals of large BAM files. Throughout these workflows, intermediate results are frequently stored in BAM format, which is typically 3–5 times smaller than the SAM format and thus helps reduce storage consumption. However, the manipulation of BAM files can introduce significant computational overhead due to compression and decompression. As a result, BAM file I/O often becomes a performance bottleneck, limiting the computational efficiency of sequencing analysis pipelines on modern multi-core systems. To address this issue, a high-performance BAM I/O library is crucial for improving the overall throughput of large-scale genomic data processing.

Since the introduction of BAM/SAM formats, a number of specialized I/O libraries have been developed. SAMtools [11], [12] is standard for processing BAM format data. It provides essential functions such as file viewing, indexing, sorting, and marking duplicates, offering foundational support for bioinformatics data analysis and laying the technical groundwork for the development of related software. HTSlib [13], separated from SAMtools, is a high-performance library that supports sequencing alignment and variant formats. Compared to the older version of SAMtools, HTSlib updates access protocols, adds support for the CRAM file format, and provides better indexing and iterators.

In addition to HTSlib and SAMtools, other tools have been proposed, trying to offer better performance or specific features. Sambamba [14], written in D, supports parallel processing for BAM file operations like sorting and indexing, and is compatible with HTSlib interfaces. Scramble [15] focuses on efficient BAM and CRAM file format conversion and uses aggressive compression to reduce BAM and CRAM file sizes. quickBAM [16] provides a framework for BAM reading based on index files, offering high speed through the use of parallel I/O. Unfortunately, only when processing sorted indexed BAM files, quickBAM can achieve better performance compared to other tools, which limits its applicability. However, with continuous updates to HTSlib, it still remains the best-performing fully functional BAM processing software known.

Consequently, commonly used BAM file processing tools are based on HTSlib. However, due to sub-optimal threading pool scheduling strategies and single-thread parsing, these tools are not able to fully utilize the computational power of modern multi-core CPUs, particularly showing poor thread scalability. For example, on a dual-socket 48-core Intel server, the latest SAMtools scales to only around 10 threads on average across typical tasks, with BAM reading using 6 threads, fixmate 16 threads and sorting 8 threads, resulting in significant underutilization of computational resources. Moreover, with the recent rapid growth of sequencing data, existing software struggles to achieve satisfactory performance. To overcome these limitations, we present RabbitBAM, a fast and scalable BAM-format file I/O library for modern multi-core platforms. By leveraging pre-parsing, parallel parsing, dedicated memory pools and lock-free queues, it achieves a significant

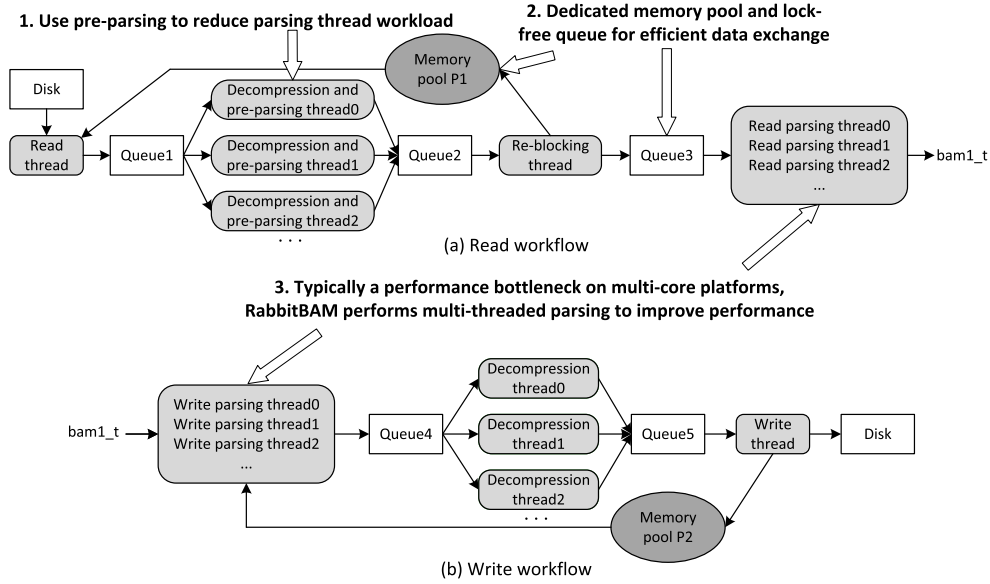


Fig. 1. Workflow of RabbitBAM.

performance improvement compared to prior implementations for both next-generation sequencing (NGS) and third-generation sequencing (TGS) data. Additionally, we implement two highly efficient tools (quality control and sorting) based on RabbitBAM as case studies.

II. METHODS

A. Overview

Fig. 1 illustrates the workflow of RabbitBAM: (a) reading, (b) writing. Both parts use a producer-consumer multi-threading model. In the reading process, a single thread reads data from disk, splits it into complete BGZF (Blocked GNU Zip Format) blocks, and puts them into *Queue1*. Multiple threads then decompress and pre-parse the data from *Queue1* in parallel, passing it to the single re-blocking thread for block splitting and merging to ensure no BAM item spans across blocks, then placing them into *Queue3*. Finally, the main thread reads binary BAM blocks from *Queue3* and hands them off to parsing threads for parallel parsing, resulting in *bam1_t* (a directly available structure) items. The writing process follows a similar pattern. Multiple parsing threads convert *bam1_t* items into binary BAM blocks, which are then compressed in parallel by compression threads, and finally, the write thread sequentially writes them to the output file.

In our multi-threading model, we use multiple threads to parallelize the most time-consuming BGZF block (de)compression tasks. Additionally, we implement an innovative pre-parsing strategy to reduce the load of the re-blocking thread and initiate multiple parsing threads for the conversion between BAM blocks and *bam1_t* items. Furthermore, we employ specifically designed memory pools and ordered lock-free queues for efficient data handling between threads.

B. BAM Reading Optimization

In the BAM file reading process, there is no dependency when decompressing BGZF blocks. Thus, most approaches typically use multiple threads to parallelize decompression tasks. However, a single BAM item might span multiple BGZF blocks (especially with the longer TGS data). Thus, parsing *block_i* might need information from the

end of *block_{i-1}*. Therefore, existing libraries usually initiate a single parsing thread to serially scan and convert BGZF blocks into *bam1_t*. On modern multi-core CPU platforms, we can often allocate dozens of threads for decompression, while a single parsing thread cannot keep up with the speed of these decompression threads. Additionally, in a multi-threaded environment, issues such as CPU resource competition, machine heat dissipation, and power limitations can lead to CPU underclocking. This phenomenon further increases the pressure of the single parsing thread, making it the performance bottleneck of the entire program. To address this problem, as shown in Fig. 1(a), we split the parsing task into two parts: re-blocking and then parsing, with detailed optimizations applied to both parts.

1) *Re-Blocking Process*: After obtaining multiple decompressed BGZF blocks, RabbitBAM initiates a re-blocking thread to re-divide and merge the blocks, ensuring no BAM item spans two or more BGZF blocks. Due to the contextual dependency of the BGZF blocks, the re-blocking operation can only be performed by a single thread. Thus, minimizing the load of this thread is crucial for optimization. RabbitBAM employs a pre-parsing strategy, attempting to parse immediately after multi-threaded decompression of each BGZF block. For better efficiency, RabbitBAM only parses the size information and some flags of each BAM item to check if parsing is successful. Then, the total size of successfully parsed BAM items (*successSize*) is recorded and passed along with the BGZF block to the re-blocking thread.

Algorithms 1 and 2 show the processing pipeline of the re-blocking thread. For each BGZF block, if *successSize* equals *BGZFBlock.length*, it indicates a complete block, which is then directly passed to the subsequent parsing threads. If *successSize* is not equal to *BGZFBlock.length*, the first *successSize* bytes of the block are sent to the parsing thread, and the remaining bytes are merged to the next BGZF block. We determine the end of a block by parsing only the size information of each BAM item. This process is applicable to both NGS and TGS data. The only difference is that for particularly long TGS BAM items, RabbitBAM will resize the block size for accommodating BAM items.

Using this pre-parsing strategy, RabbitBAM is able to perform some preliminary parsing tasks within the decompression threads in parallel, significantly reducing the load of the re-blocking thread and preventing it from becoming a performance bottleneck.

Algorithm 1: Processing Logic of the Re-Blocking Thread.

Input: *Queue2*, which holds decompressed BGZF blocks
Output: Processed block stream to parsing threads

```

1: while true do
2:   nowBlock  $\leftarrow$  new assignBlock()
3:   BGZFBlock  $\leftarrow$  getBlockFromQueue2()
4:   if BGZFBlock == nullptr then
5:     break
6:   end if
7:   successSize  $\leftarrow$  BGZFBlock.bamSize
8:   if successSize  $\neq$  BGZFBlock.length then
9:     copy(nowBlock, BGZFBlock, successSize)
10:    nowBlock.length  $+=$  successSize
11:    pushToParser(nowBlock)
12:    restSize  $\leftarrow$  BGZFBlock.length - successSize
13:    while restSize  $\neq$  0 do
14:      nowBlock  $\leftarrow$  new assignBlock()
15:      copy(nowBlock, BGZFBlock +
16:        successSize, restSize)
17:      nowBlock.length  $+=$  restSize
18:      BGZFBlock  $\leftarrow$  getBlockFromQueue2()
19:      copy(nowBlock +
20:        nowBlock.length, BGZFBlock, BGZFBlock.length)
21:      nowBlock.length  $+=$  BGZFBlock.length
22:      lastPos  $\leftarrow$  findLastBAM(nowBlock)
23:      restSize  $\leftarrow$  nowBlock.length - lastPos
24:      copy(BGZFBlock, nowBlock + lastPos, restSize)
25:      successSize  $\leftarrow$  0
26:      nowBlock.length = lastPos
27:      pushToParser(nowBlock)
28:    end while
29:  else
30:    copy(nowBlock, BGZFBlock, successSize)
31:    nowBlock.length  $+=$  successSize
32:    pushToParser(nowBlock)
33:  end if
34: end while

```

2) *Parsing Process*: In Section II-B1, we minimize the load of the re-blocking thread, leaving the detailed parsing process to the parsing threads. The complexity of the parsing threads is much greater than that of the re-blocking thread, as the latter only needs to parse length information, while the former must fully parse and organize the data into bam1_t format. Therefore, parallel acceleration with many CPU cores is a straightforward but effective way to improve the performance of the parsing process. Since cross-block issues have already been resolved in the re-blocking thread, the parsing threads can easily achieve block-based parallelism, with the only requirement being to maintain the order among BAM items. In order to achieve this, RabbitBAM divides the parsing process into multiple rounds. In each round, the main thread fetches blocks from the re-blocking thread and distributes them to multiple parsing threads for parallel processing. Finally, the main thread merges the resulting bam1_t items and passes them to subsequent tasks. By utilizing static task division based on thread_id, RabbitBAM ensures the order of BAM items while maintaining high efficiency.

3) *Dedicated Memory Pools and Lock-Free Queues*: Based on Fig. 1, RabbitBAM processes data in streaming fashion and uses a total of five queues: two single-in-multiple-out queues, two multiple-in-single-out queues, and one single-in-single-out queue, which means our framework involves extensive data handling and memory operations.

Algorithm 2: Finding the Last Unbroken BAM Item.

Input: *nowBlock*, the current block being processed
Output: *lastPos*, last complete BAM record position

```

1: function FINDLASTBAM(nowBlock)
2:   while the first BAM item is too long do
3:     BGZFBlock  $\leftarrow$  getBlockFromQueue2()
4:     resize(nowBlock)
5:     copy(nowBlock +
6:       nowBlock.length, BGZFBlock, BGZFBlock.length)
7:     nowBlock.length  $+=$  BGZFBlock.length
8:   end while
9:   lastPos  $\leftarrow$  0
10:  while not end of nowBlock do
11:    currentPos  $\leftarrow$  find next BAM item end position
12:    if BAM item at currentPos is unbroken then
13:      lastPos  $\leftarrow$  currentPos
14:    end if
15:  end while
16:  return lastPos
17: end Function

```

Thus, efficiently handling these queues, especially in a multi-threaded environment, is crucial. To address these challenges, we design an efficient lock-free queue and reduce the overhead of enqueueing operations by limiting the maximum number of elements allowed in the queue. Additionally, we also adopt a memory pool to reduce the overhead of redundant memory allocations. By integrating the use of lock-free queues and memory pools, we can control the maximum number of elements in the queue by simply limiting the memory pool size.

The memory pool *PI* (consisting of 256 memory blocks of 64 KB each by default) is used between the read thread and the re-blocking thread. We ensure that only the read thread fetches (single-out) data while the re-blocking thread returns (single-in) data, see Fig. 1. The life-cycle of a memory block in *PI* is as follows: used by the read thread to read data from disk, passed to decompression threads for decompression, passed to the re-blocking thread for block copying, and finally returned to the memory pool *PI*. Since the re-blocking thread may resize the memory block when processing long TGS data, *PI* does not cover the entire re-blocking process. Additionally, since *PI* is single-in-single-out, RabbitBAM can efficiently manage the memory pool using only two atomic variables, avoiding the overhead of competition in a multi-threaded environment.

RabbitBAM designs *Queue1* and *Queue2* as efficient ordered lock-free queues. Taking *Queue1* as an example, Algorithm 3 details the dequeuing and enqueueing operations. *qHead* and *qTail* represent the head and tail of the queue, respectively. *isComplete* indicates whether the read thread has finished, and *queueSize* is the predefined capacity of the circular buffer. A single read thread performs enqueueing operations, updating *qTail* and *isComplete*. Multiple decompression threads perform dequeuing operations in parallel, reading *qTail* and *isComplete*, while also accessing *qHead* for both reading and writing. Since *qHead* is written concurrently, it is declared as an atomic variable to ensure all decompression threads observe its latest value. In contrast, *qTail* and *isComplete* are updated exclusively by the reading thread and are therefore defined as non-atomic variables for improved performance. Although this configuration may result in decompression threads observing outdated values of *qTail* and *isComplete*, the queue can still function correctly through some redundant operations.

Specifically, when multiple decompression threads fetch data from *Queue1*, they first read the current value of *qTail* into a temporary variable, *tmpVal*. Next, a CAS (Compare-And-Swap) operation is used

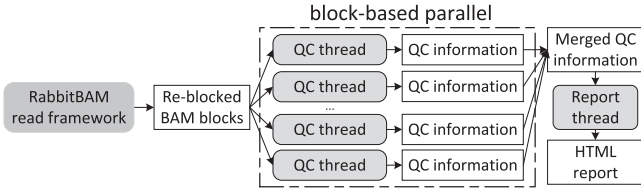


Fig. 2. Workflow of RabbitBAM-QC.

to compare $tmpVal$ with $qHead$. Because $qTail$ is non-atomic and may be outdated, the comparison can lead to the following cases:

1. $qHead == tmpVal$:
 - If $tmpVal$ is up-to-date, this indicates the queue is empty and the decompression threads should wait. During this period, $isComplete$ must be checked to avoid unnecessary blocking.
 - If $tmpVal$ is outdated, the decompression threads also proceed to Lines 5–9 to refresh $tmpVal$ and re-evaluate. Notably, the check in Line 6 is safe: before updating $isComplete$, the read thread performs a release memory fence to synchronize the latest $qTail$ across all decompression threads (Line 21). Since this synchronization occurs only once at the end of the read thread, the performance impact is negligible.
2. $qHead \neq tmpVal$: The CAS operation sets $tmpVal$ to $qHead$ and exits the loop.
 - If $qHead < tmpVal$, the data at position $qHead$ is valid. A CAS operation is then used to acquire this position and update $qHead$, and the corresponding data is returned.
 - If $qHead > tmpVal$, it indicates that $tmpVal$ is outdated. The decompression threads then return to Line 3 to refresh $tmpVal$ and recheck. Theoretically, the cases $qHead == tmpVal$ and $qHead > tmpVal$ could be handled in the same way, but CAS operations only compare for equality. Therefore, they are treated separately in the algorithm.

In summary, this dequeuing process ensures that the $qTail$ used is always less than or equal to the actual queue tail. By incorporating redundant operations and stricter logic, we ensure the correctness of queue behavior. In addition, we minimize the use of atomic variables and avoid locks whenever possible, achieving highly efficient multi-threaded dequeuing.

To further improve the performance of enqueueing operations, we set the maximum size of *Queue1* to twice the size of *P1*. When the read thread enqueuees data into *Queue1*, as shown in Algorithm 3, only lines 17 and 18 are needed. Since the enqueueed data resides in memory pool and the size of the memory pool is smaller than the queue size, so there is no need to check if the queue is full during the enqueueing operation. Additionally, we use a circular queue to further reduce memory usage.

C. BAM Writing Optimization

Fig. 1(b) shows the writing process of RabbitBAM. First, we parallelize the conversion of `bam1_t` to binary BAM blocks and use similar optimization methods to the read parsing part (Section II-B2) to achieve enhanced efficiency and ensure data remains ordered. Additionally, we parallelize the computationally intensive BGZF block compression using two ordered lock-free queues, *Queue4* and *Queue5*, along with memory pool *P2*, enabling efficient multi-threaded data synchronization and handling. Finally, a single write thread sequentially writes the compressed BGZF blocks to the disk. We further streamline the data enqueueing logic for the lock-free queues by constraining the sizes of *P2*, *Queue4*, and *Queue5*.

Algorithm 3: Dequeuing/Enqueueing for Single-in-Multiple-Out Ordered Lock-Free Queue.

Input: $qTail$, the tail position of the queue; $qHead$, the head position of the queue; $isComplete$, flag indicating if the read thread is complete; $queueData$, array of data in the queue; $queueSize$, size of the queue; $data$, the data to be enqueueed

Output: The dequeued or enqueueed result

```

1: /* Decompression thread dequeuing operation */
2: while true do
3:   tmpVal ← qTail
4:   while qHead.CAS(tmpVal, tmpVal) do
5:     sleep()
6:     if isComplete and qHead.load() == qTail then
7:       return nullptr
8:     end if
9:     tmpVal ← qTail
10:  end while
11:  if tmpVal < qTail and qHead.CAS(tmpVal, tmpVal + 1) then
12:    return queueData[tmpVal%queueSize]
13:  end if
14: end while
15:
16: /* Read thread enqueueing operation */
17: queueData[qTail%queueSize] ← data
18: qTail ← qTail + 1
19:
20: /* Read thread exiting operation */
21: atomic_thread_fence(memory_order_release)
22: isComplete ← true

```

D. Design of Case Studies

To validate the performance of the RabbitBAM I/O library, we design two case studies: BAM quality control (RabbitBAM-QC) and sorting (RabbitBAM-Sort).

BAM quality control provides detailed data statistics like read count, read length, and base quality distribution, enabling targeted filtering and trimming for downstream tasks. Since I/O is often the bottleneck in these linear-complexity operations, we developed RabbitBAM-QC using RabbitBAM, with its workflow shown in Fig. 2. RabbitBAM-QC reads re-blocked BAM data, then uses parallel processing to parse and perform quality control on each block. During this process, each thread maintains its own statistical data, which is aggregated into the main thread once all blocks have been processed. Finally, RabbitBAM-QC generates a visual HTML report.

BAM file sorting is an essential task before duplicate marking, variant calling, and other downstream processes. It organizes BAM items by the reference genome's position, significantly improving data access efficiency. Since sorting is a typical offline processing task which requires reading all the data before processing, it normally demands substantial memory. Therefore, RabbitBAM-SORT employs a hybrid internal-external sorting strategy to limit memory usage while maximizing efficiency. Fig. 3 shows the processing workflow of RabbitBAM-SORT. When the whole decompressed BAM file can fit into memory (Fig. 3(a)), we use in-memory sorting (parallel merge sort) and utilize RabbitBAM to accelerate file reading and writing. When memory is insufficient (Fig. 3(b)), we use external sorting and store the intermediate BAM files on the disk. First, we use RabbitBAM to split the input BAM file into multiple chunks according to the memory limit and then process each chunk one by one. For each chunk, we launch

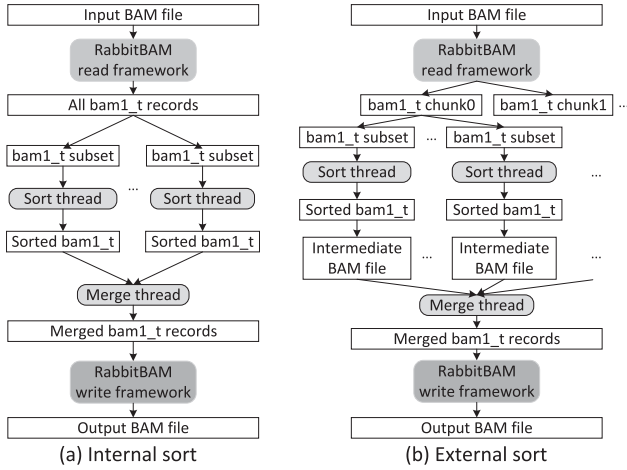


Fig. 3. Workflow of RabbitBAM-SORT.

TABLE I
SPECIFICATION OF THE UTILIZED HARDWARE PLATFORMS

	Intel Server	AMD Server
CPU	Xeon Platinum 8375 * 2	EPYC 7T83 * 2
Cores	64	128
RAM	256 GB	256 GB
SSD	Samsung 970 2T * 4 (RAID0)	WD SN640 7.68T
OS	Rocky Linux 8.6	Rocky Linux 8.6
Compiler	GCC 8.5.0	GCC 8.5.0

multiple sorting threads to achieve local order in parallel and write the intermediate BAM files to disk. After all chunks are sorted, we use a multi-way merge sort strategy to combine all small BAM files into a globally sorted dataset, and finally, use RabbitBAM to perform the write operation. For the intermediate BAM file writing, we use serial APIs because this process is implemented as file-based parallelism, with each file being handled by a single thread. In addition, when merging intermediate BAM files, similar to SAMtools, RabbitBAM-SORT uses serial APIs to open all the files simultaneously (with the number potentially far exceeding the CPU core count).

III. RESULTS

A. Overview

The experiment consists of two parts. First, in Section III-B and III-C, we compare the performance of the basic I/O operations, evaluating RabbitBAM against SAMtools (v1.20), Sambamba (v1.0.1) and quickBAM (v1.0.3) in read-only and read-and-write modes, if supported by the tool. Additionally, to evaluate the effectiveness of the RabbitBAM, in Section III-D, we compare the performance of BAM quality control and sorting case studies implemented with RabbitBAM against SAMtools and Sambamba. Since SAMtools is based on HTSLib and their performance is identical, HTSLib is not listed separately.

Table I shows the experimental platforms, which include a 64-core Intel server and a 128-core AMD server. Both are equipped with high-speed SSDs running under Linux and utilize GCC 8.5.0. Additionally, we ensure that RabbitBAM's output has the same MD5 checksum as SAMtools (except for the header information, which contains the command used to generate the file, such as "samtools view in.bam").

Table II presents the datasets used for testing, which include various types of BAM data with different lengths, sizes, and sequencing

TABLE II
DESCRIPTION OF UTILIZED DATASETS

NCBI accession	Abbr.	#Reads	Average length	Data size
SRX4728450	D1	177,797,579	126	9.5G
SRX4728514	D2	1,212,177,023	151	85.0G
SRX13341148	D3	184,455	12,635	4.2G

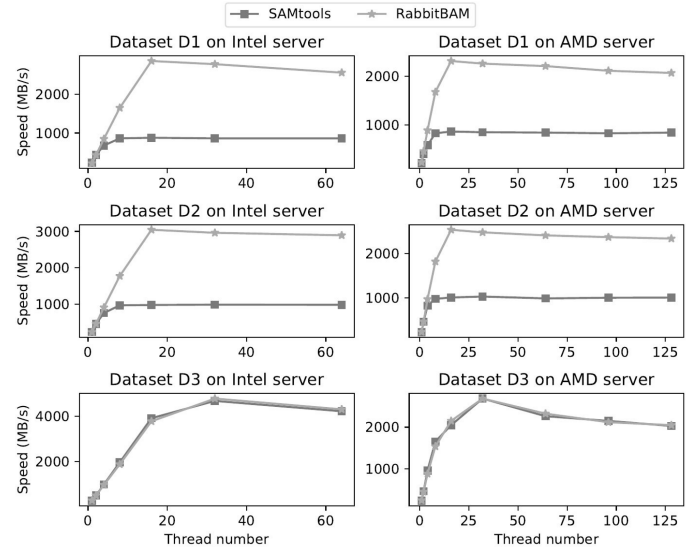


Fig. 4. Performance comparison of read-only mode.

technologies. Specifically, D1 is a human whole-exome NGS dataset with 880x depth; D2 is a human whole-genome NGS dataset derived from tumor tissue with 60x sequencing depth; D3 is a *Streptococcus delphini* strain TGS dataset. All BAM files are compressed with the default BGZF compression level (6).

B. Read-Only Performance

For the read-only mode, to eliminate the impact of additional operations, we use the "count line" function of each tool to evaluate data reading performance. Fig. 4 compares the performance between RabbitBAM and SAMtools across different thread numbers, platforms, and datasets. Under this mode, we do not compare Sambamba because it does not offer a "count line" function. In addition, we separately compare the performance of quickBAM and RabbitBAM on indexed BAM files, as quickBAM does not support general-purpose unindexed BAM files.

For NGS data (D1 and D2), RabbitBAM achieves speedups of 2.7-3.3x over SAMtools for read operations. Due to optimizations in re-blocking and parsing tasks, RabbitBAM demonstrates better thread scalability, maintaining a good speedup even up to 16 threads. However, due to the high load on the single parsing thread, SAMtools shows no significant performance gains beyond 8 threads.

For TGS data (D3), BAM items are typically much longer, which means there are fewer items in a file of the same size. For example, the D1 dataset has an average of 18 million items per GB, while D3 has only 44 thousand. This reduces the load of the parsing thread, as it only needs to process the metadata of each BAM item rather than the entire sequence data. This data characteristic makes RabbitBAM's optimizations for the parsing process less effective, with decompression remaining the primary performance bottleneck. To validate this hypothesis, we restructure the original queue-based streaming pipeline by

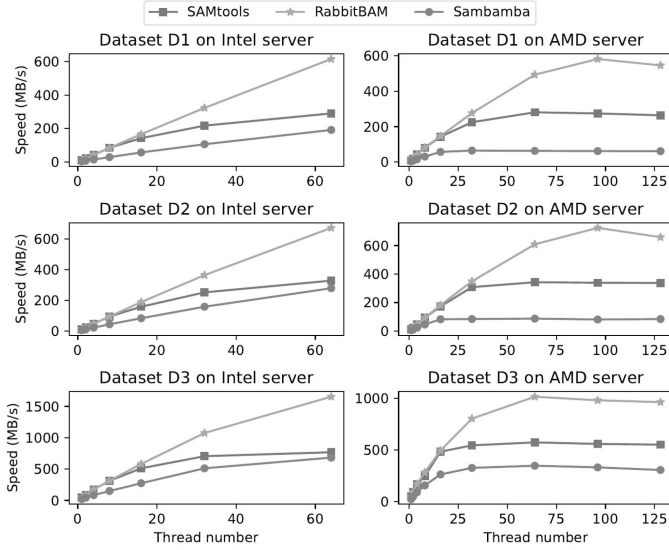


Fig. 5. Performance comparison of read-and-write mode.

decoupling decompression and parsing operations into serial execution phases while eliminating potential I/O bottlenecks. Profiling experiments on the Intel server (using 48 threads) show that for D3 (TGS) data processing, decompression achieves 5 GB/s throughput, compared to 7 GB/s for parsing, with decompression being the system bottleneck. In contrast, D1 (NGS) reverses this pattern (4.8 GB/s decompression, 2.6 GB/s parsing), highlighting the higher computational complexity of NGS parsing. RabbitBAM only optimizes parsing, using the same libdeflate decompression engine as SAMtools. Consequently, there is a significant performance improvement on NGS data, while remaining similar for TGS data. Additionally, we observe that with the D3 dataset, both tools achieve a performance of 5 GB/s on the Intel server but only 2.5 GB/s on the AMD server. This difference is because they reach the maximum disk read speed, and the two platforms have different disk speeds, leading to a significant performance gap.

While the previous evaluation focuses on general-purpose unindexed BAM files, some downstream workflows (e.g., variant calling) use sorted, indexed BAM files. quickBAM is specifically optimized for such scenarios: it leverages index information to enable multi-threaded reading from multiple positions, achieving near-linear scalability. On the Intel server, quickBAM reaches a read throughput of up to 4.5 GB/s with 64 threads, about 1.5x faster than RabbitBAM (limited by parsing, scaling up to 3 GB/s with 16 threads) on NGS data. We do not evaluate quickBAM on TGS data or in write mode, as it either fails or lacks support. RabbitBAM, by contrast, is a general-purpose library for all stages, such as fixmate, sorting, indexing, and markup. It treats indexed BAM files as regular input and does not currently utilize index information for parallel reading, leaving room for future performance improvements.

C. Read-and-Write Performance

For the read-and-write mode, we use the “view” function to compare performance. In the “view” function, each tool decompresses and parses the BAM file into a usable data structure (e.g., `bam1_t`), performs simple statistical operations, and then compresses the data back into a BAM file before writing it to disk. Fig. 5 shows the performance comparison for this process.

For NGS data (D1 and D2), RabbitBAM achieves approximately a 2x speedup compared to SAMtools and 3-9x speedups compared to

TABLE III
PERFORMANCE COMPARISON OF BAM FILE QUALITY CONTROL AMONG RABBITBAM-QC, SAMTOOLS, AND SAMBAMBA ON THE INTEL SERVER

Tool	Runtime (s)	Speedup	Threads
Dataset D1			
RabbitBAM-QC	6.8	-	16
SAMtools flagstat	11.4	1.8	8
Sambamba flagstat	10.8	1.6	8
Dataset D2			
RabbitBAM-QC	50.4	-	24
SAMtools flagstat	87.9	1.7	16
Sambamba flagstat	70.9	1.4	16

TABLE IV
PERFORMANCE COMPARISON OF BAM FILE SORTING AMONG RABBITBAM-SORT, SAMTOOLS, AND SAMBAMBA ON THE INTEL SERVER

Tool	Runtime (s)	Speedup	Threads
Dataset D1, internal sorting			
RabbitBAM-SORT	36.8	-	64
SAMtools sort	74.9	2.0	48
Sambamba sort	90.1	2.4	64
Dataset D2, external sorting			
RabbitBAM-SORT	410.7	-	48
SAMtools sort	621.2	1.5	48
Sambamba sort	754.3	1.8	48

Sambamba. Compared to the read-only mode, RabbitBAM achieves better thread scalability (scaling up to 64 or even more threads) due to the addition of parallel compression tasks.

For TGS data (D3), SAMtools uses a single thread to handle both the reading and writing parsing, which significantly increases the load of the parsing task and limits the efficient utilization of multiple CPU cores. In contrast, RabbitBAM optimizes the parsing task for both reading and writing, resulting in better performance and achieving speedups of 1.8-2.9x. Additionally, the performance difference between the Intel and AMD platforms (1.5 GB/s vs. 1 GB/s peak) is still due to disk speed.

D. Case Studies

For the quality control case study, we compare the performance of RabbitBAM-QC with the “flagstat” function of SAMtools and Sambamba. All tools offer similar functionality and are primarily limited by BAM read performance, making them ideal for testing the efficiency of I/O framework. Table III presents a performance comparison across different datasets on the Intel server, showing the optimal performance and corresponding thread number for 1-64 threads. RabbitBAM achieves 1.4-1.8x speedups, reaching GB/s level BAM quality control speed with fewer than 20 threads. Additionally, RabbitBAM-QC provides a visual HTML report which can highlight the data quality issues more intuitively.

Table IV presents a performance comparison of BAM sorting among RabbitBAM-SORT, SAMtools and Sambamba. We use the smaller D1 dataset (9.5 GB) to test in-memory sorting performance and the larger D2 dataset (85 GB) to test external sorting performance. RabbitBAM-SORT achieves a sorting speed of 260 MB/s using 64 threads for in-memory sorting, resulting in a speedup of at least 2x compared to SAMtools and Sambamba. When performing external sorting, RabbitBAM achieves 1.5-1.8x speedups. This is because the bottleneck

TABLE V
PERFORMANCE COMPARISON OF VARIANT CALLING PIPELINES BASED ON
DIFFERENT BAM I/O LIBRARIES ON THE INTEL SERVER

Tool	Runtime (s)	Speedup
RabbitBAM-total pipeline	6,265	-
SAMtools-total pipeline	6,935	1.11
Sambamba-total pipeline	7,355	1.17
RabbitBAM-BAM process	2,927	-
SAMtools-BAM process	3,597	1.23
Sambamba-BAM process	4,017	1.37

lies in the single-threaded reading of intermediate BAM files, which cannot be parallelized because too many files are open simultaneously. In addition, the theoretical space complexity of the sorting algorithms used by these tools corresponds to the size of the SAM records obtained after decompressing the BAM file. To avoid exceeding the system's memory limit, they specify the amount of memory available to the program at runtime. If the size of the SAM data surpasses this threshold, external sorting is performed; otherwise, internal sorting is used.

Beyond evaluating sorting performance in isolation, we further validate RabbitBAM by integrating it into a somatic variant calling pipeline. The workflow employs Strobealign [17] for sequence alignment, followed by BAM file sorting, duplicate marking, and variant calling with RabbitVar [18]. The input consists of whole-genome sequencing data from paired normal (SRX4728513) and tumor (SRX4728514, corresponding to D2 in Table II) tissue samples from the same patient, provided in FASTQ format with 60× sequencing depth. Table V shows the performance comparison of different BAM I/O libraries. The RabbitBAM-enhanced pipeline achieves an overall performance improvement of 11–17%, with a 1.23–1.37× speedup specifically in the BAM processing stage.

IV. CONCLUSION

This paper introduces RabbitBAM, an efficient BAM file I/O library. In RabbitBAM, we address the single-threaded parsing bottleneck by using pre-parsing and parallel parsing strategies. Additionally, we reduce the multi-threaded data handling overhead through the use of dedicated lock-free queues and memory pools. As a result, RabbitBAM offers superior thread scalability and achieves significantly improved performance over other tools on modern multi-core platforms. In summary, RabbitBAM elevates BAM file processing speed on multi-core platforms to a new level, providing a more efficient I/O engine for the sequencing data analysis workflow. In the future, we plan to further enhance the usability of RabbitBAM, explore more efficient compression

engines to surpass the performance of the currently used libdeflate, and leverage index information to further improve the efficiency of processing indexed BAM files.

REFERENCES

- [1] B. E. Slatko, A. F. Gardner, and F. M. Ausubel, "Overview of next-generation sequencing technologies," *Curr. Protoc. Mol. Biol.*, vol. 122, no. 1, 2018, Art. no. e59.
- [2] S. L. Amarasinghe, S. Su, X. Dong, L. Zappia, M. E. Ritchie, and Q. Gouil, "Opportunities and challenges in long-read sequencing data analysis," *Genome Biol.*, vol. 21, no. 1, 2020, Art. no. 30.
- [3] J. C. Wooley and Y. Ye, "Metagenomics: Facts and artifacts, and computational challenges," *J. Comput. Sci. Technol.*, vol. 25, no. 1, pp. 71–81, 2010.
- [4] S. Goodwin, J. D. McPherson, and W. R. McCombie, "Coming of age: Ten years of next-generation sequencing technologies," *Nat. Rev. Genet.*, vol. 17, no. 6, pp. 333–351, 2016.
- [5] B. Schmidt and A. Hildebrandt, "Next-generation sequencing: Big data meets high performance computing," *Drug Discov. Today*, vol. 22, no. 4, pp. 712–717, 2017.
- [6] S. Chen, Y. Zhou, Y. Chen, and J. Gu, "Fastp: An ultra-fast all-in-one FASTQ preprocessor," *Bioinformatics*, vol. 34, no. 17, pp. i884–i890, 2018.
- [7] H. Li, "Minimap2: Pairwise alignment for nucleotide sequences," *Bioinformatics*, vol. 34, no. 18, pp. 3094–3100, 2018.
- [8] S. Kim et al., "Strelka2: Fast and accurate calling of germline and somatic variants," *Nat. Methods*, vol. 15, no. 8, pp. 591–594, 2018.
- [9] M. I. Love, S. Anders, V. Kim, and W. Huber, "RNA-Seq workflow: Gene-level exploratory analysis and differential expression," *F1000Research*, vol. 4, 2015, Art. no. 1070.
- [10] M. Pertea, G. M. Pertea, C. M. Antonescu, T.-C. Chang, J. T. Mendell, and S. L. Salzberg, "Stringtie enables improved reconstruction of a transcriptome from RNA-Seq reads," *Nat. Biotechnol.*, vol. 33, no. 3, pp. 290–295, 2015.
- [11] P. Danecek et al., "Twelve years of samtools and bcftools," *Gigascience*, vol. 10, no. 2, 2021, Art. no. giab008.
- [12] H. Li et al., "The sequence alignment/map format and samtools," *bioinformatics*, vol. 25, no. 16, pp. 2078–2079, 2009.
- [13] J. K. Bonfield et al., "Htslib: C library for reading/writing high-throughput sequencing data," *Gigascience*, vol. 10, no. 2, 2021, Art. no. giab007.
- [14] A. Tarasov, A. J. Vilella, E. Cuppen, I. J. Nijman, and P. Prins, "Sambamba: Fast processing of NGS alignment formats," *Bioinformatics*, vol. 31, no. 12, pp. 2032–2034, 2015.
- [15] J. K. Bonfield, "The scramble conversion tool," *Bioinformatics*, vol. 30, no. 19, 2014, Art. no. 2818.
- [16] A. Pitman, X. Huang, G. T. Marth, and Y. Qiao, "quickBAM: A parallelized BAM file access API for high-throughput sequence analysis informatics," *Bioinformatics*, vol. 39, no. 8, 2023, Art. no. btad463.
- [17] K. Sahlin, "Strobealign: Flexible seed size enables ultra-fast and accurate read alignment," *Genome Biol.*, vol. 23, no. 1, 2022, Art. no. 260.
- [18] H. Zhang et al., "RabbitVar: Ultra-fast and accurate somatic small-variant calling on multi-core architectures," *bioRxiv*, pp. 2023–1, Jan. 2023.